



HACKTHEBOX



Login Simulator

15th April 2022 / Document No. D22.102.303

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Medium

Classification: Official

Synopsis

The challenge comes with the source code of the binary and analyzing it we first can see 1 bug in `getInput` function. For the counter it uses signed char, which range is from -127 to 127, but our max size for the username length that we can specify is 0x80 (128) meaning that there is a buffer underflow there if we set the username size to 128. When it reaches 127 after incrementing it again it will go back to -127 and then start from there up. We can abuse this to overwrite a return address with our rop chain.

Canary is also there which would be a problem, but there is another bug in the same function where it will check if the character is a space and if it is it will just do , meaning that it will skip over the space from the user input, but, it will also keep incrementing the counter. With that we can just send a bunch of spaces to skip over the canary and get straight to the return address and write our rop chain there.

```
void getInput(char *buf, int len) {
    char tmp;
    char i; // signed char, vuln because 'len' can go to 128
    for (i = 0; i < len; i++) {
        if (read(0, &tmp, 1) <= 0) {
            break;
        }
        // logic bug where it will skip the space but also increment 'i'
        if (tmp == ' ') {
            continue;
        }
    }
}
```

```

        if (tmp == '\n') {
            break;
        }
        buf[i] = tmp;
    }
}

```

We also need a way to get leaks somehow as there is PIE. From the functionality of and functions we see that while registering a username it asks us for the length of it, which later in function it's used in the function call.

Also we see that the inside the main function is not initialized to NULL, meaning somewhere in it's range 0-0x80 will be some junk, some of that junk is a pie address and a libc address.

Solution

Now to actually leak either of those addresses we can, firstly, register a user with the length to be `offset to the address we want to leak + 1`. Example, if inside the `username` variable is a libc address at the offset 0x70 from the starting byte, we can register a username with the length of 0x71 and set the username to be `'A' * 0x70`. Then we can abuse the `_login` functionality where it will call with the length we specified. So it will call `strcmp(buf, username, 0x71)`, where the `username` is `'A' * 0x70` specified earlier and `buf` is also `'A' * 0x70`. Now `strcmp` is gonna go and compare the 0x71 bytes where the first 0x70 are correct but the 0x71st is not, 0x71st byte is the first byte of the address we want to leak.

So we loop through 0x00-0xff bytes trying to login with `'A'*0x70 + guess_byte` and when we get the correct login message we know we have the correct lsb byte of the address. We now just do that for the next 5 bytes of the address, register a new username with the length of 0x72 this time, and this time as we know the lsb byte we can just include it in our payload and guess the second byte of the address, and this way we can do a byte by byte bruteforce of the address we want to leak.

If we just set a breakpoint in gdb somewhere inside `main` and check the contents of the uninitialized `username` variable we can see that at offset 0x70 there is a libc address we can leak the way just explained before this. And after we get that libc leak we can abuse a buffer underflow inside function and also abuse the 'byte skipping' bug, also inside function, to skip over the canary and then overwrite the return address and spawn a shell.